Travaux Dirigés Avancés
Framework PHP – Laravel

Niveau avancé à très avancé

Licence 3 SIO – RAZAFINDRAIBE Fabrice

Projet fil rouge

Application de gestion de stock, commandes et services associés.

$$\boxed{HTTP \rightarrow Middleware \rightarrow Controller \rightarrow Service \rightarrow Domain \rightarrow Persistence}$$

puis :

$$\boxed{Events + Queues + Cache + API + Tests + Déploiement}$$

ESMIA – 2026

Table des matières

1 Organisation de la série avancée **2**

2 TD 16 – Middleware personnalisés et pipeline HTTP **3**
2.1 1. Rappel conceptuel . . . 3
2.2 2. Créer un middleware de mesure . . . 3
2.3 3. Utiliser le middleware . . . 4
2.4 4. Ajouter un header à la réponse . . . 4
2.5 5. Middleware paramétré . . . 4
2.6 6. Alias . . . 5
2.7 7. Observer l'ordre . . . 5
2.8 8. Travail autonome . . . 6

3 TD 17 – Service Container et Service Providers **7**
3.1 1. Cas d'étude : paiement . . . 7
3.2 2. Première implémentation . . . 7
3.3 3. Binding . . . 8
3.4 4. Injection . . . 8
3.5 5. Seconde implémentation . . . 8
3.6 6. bind ou singleton ? . . . 8
3.7 7. Résolution manuelle . . . 9
3.8 8. Remplacement en test . . . 9
3.9 9. Défi . . . 9

4 TD 18 – Events, Listeners et Observers **10**
4.1 1. Problème . . . 10
4.2 2. Créer un Event . . . 10
4.3 3. Déclencher . . . 10
4.4 4. Listener . . . 11
4.5 5. Plusieurs listeners . . . 11
4.6 6. Observer . . . 11
4.7 7. Comparaison . . . 12
4.8 8. Travail autonome . . . 12

5 TD 19 – Jobs, Queues et traitement asynchrone **13**
5.1 1. Problème . . . 13
5.2 2. Créer un Job . . . 13
5.3 3. Dispatcher . . . 14
5.4 4. Driver database . . . 14
5.5 5. Worker . . . 14
5.6 6. Provoquer un échec . . . 14
5.7 7. Méthode failed() . . . 15
5.8 8. Analyse . . . 15
5.9 9. Travail autonome . . . 15

6 TD 20 – Cache et optimisation des performances **16**
6.1 1. Dashboard . . . 16
6.2 2. Première version . . . 16
6.3 3. Cache . . . 16
6.4 4. Invalidation . . . 17
6.5 5. N+1 . . . 17
6.6 6. Limiter les colonnes . . . 17
6.7 7. Index SQL . . . 18

6.8 8. Travail autonome 18

7 TD 21 – Sécurité applicative Laravel 19
7.1 1. Audit initial 19
7.2 2. Validation 19
7.3 3. Mass assignment 19
7.4 4. XSS 20
7.5 5. CSRF 20
7.6 6. Tri dynamique dangereux 20
7.7 7. Upload sécurisé 21
7.8 8. Rate limiting 21
7.9 9. URL signée 21
7.10 10. Travail autonome 22

8 TD 22 – API authentifiée par token 23
8.1 1. Préparation 23
8.2 2. Création d'un token 23
8.3 3. Appel HTTP 23
8.4 4. Route protégée 23
8.5 5. Vérifier une capacité 24
8.6 6. Révoquer 24
8.7 7. Scénarios 24
8.8 8. Défi 24

9 TD 23 – Concurrence et cohérence du stock 25
9.1 1. Situation 25
9.2 2. Code naïf 25
9.3 3. Transaction avec verrou 25
9.4 4. Analyse temporelle 26
9.5 5. Quantité supérieure à 1 26
9.6 6. Travail autonome 27

10 TD 24 – Idempotence et robustesse d'une API 28
10.1 1. Situation 28
10.2 2. Header 28
10.3 3. Table 28
10.4 4. Algorithme 29
10.5 5. Questions difficiles 29
10.6 6. Défi 29

11 TD 25 – Tests avancés, Fakes et Mocks 30
11.1 1. Queue Fake 30
11.2 2. Vérifier le contenu 30
11.3 3. Event Fake 30
11.4 4. Notification Fake 31
11.5 5. Remplacer PaymentGateway 31
11.6 6. Tester rollback 31
11.7 7. Travail autonome 32

12 TD 26 – Architecture applicative et refactoring 33
12.1 1. Contrôleur problématique 33
12.2 2. Architecture cible 33
12.3 3. DTO 33
12.4 4. Value Object Quantity 34
12.5 5. Action 34

12.6 6. Contrôleur final . . . 35
12.7 7. Discussion Repository . . . 35
12.8 8. Travail autonome . . . 35

13 TD 27 – Commandes Artisan et Scheduler 36
13.1 1. Créer une commande . . . 36
13.2 2. Signature . . . 36
13.3 3. Exécution . . . 36
13.4 4. Scheduler . . . 37
13.5 5. Tester localement . . . 37
13.6 6. Travail autonome . . . 37

14 TD 28 – Déploiement Laravel sous Linux 38
14.1 1. Architecture . . . 38
14.2 2. Dépendances production . . . 38
14.3 3. Environnement . . . 38
14.4 4. Clé d'application . . . 38
14.5 5. Permissions . . . 39
14.6 6. Caches . . . 39
14.7 7. Migration . . . 39
14.8 8. Configuration Nginx simplifiée . . . 39
14.9 9. Worker . . . 40
14.1010. Logs . . . 40
14.1111. Travail autonome . . . 40

15 TD 29 – Broadcasting et temps réel 41
15.1 1. Scénario . . . 41
15.2 2. Event broadcast . . . 41
15.3 3. Autorisation du canal . . . 41
15.4 4. Analyse . . . 42

16 TD 30 – Incident de production simulé 43
16.1 1. Situation . . . 43
16.2 2. Méthode de diagnostic . . . 43
16.3 3. Dashboard lent . . . 43
16.4 4. Stock négatif . . . 44
16.5 5. Commande doublée . . . 44
16.6 6. Facture absente . . . 44
16.7 7. Accès administration . . . 44
16.8 8. API 500 . . . 44
16.9 9. Données obsolètes . . . 44
16.1010. Livrable final . . . 45

17 Projet de synthèse avancé 46

18 Architecture cible 46

19 Proposition de barème 47

20 Questions possibles de soutenance 47

21 Synthèse finale 48

1. Organisation de la série avancée

Cette série intervient après les TD portant sur :

— CRUD et validation ;

— relations Eloquent ;

— requêtes avancées et problème N+1 ;

— authentification et autorisation ;

— services et injection de dépendances ;

— transactions ;

— API REST ;

— tests automatisés.

Le niveau de guidage diminue fortement.

L'étudiant doit maintenant être capable de lire la documentation, analyser un problème, proposer plusieurs solutions, justifier son choix et vérifier son comportement par des tests.

TD	Thème	Niveau
16	Middleware et pipeline HTTP	Avancé
17	Service Container et Providers	Avancé
18	Events, Listeners et Observers	Avancé
19	Jobs et Queues	Avancé
20	Cache et performance	Avancé
21	Sécurité applicative	Avancé +
22	API authentifiée par token	Avancé +
23	Concurrence et cohérence des stocks	Très avancé
24	Idempotence d'une API	Très avancé
25	Tests avancés et doubles de test	Très avancé
26	Architecture et refactoring	Très avancé
27	Artisan et Scheduler	Très avancé
28	Déploiement Linux	Très avancé
29	Temps réel et Broadcasting	Option expert
30	Incident de production simulé	Synthèse expert

2. TD 16 – Middleware personnalisés et pipeline HTTP

Objectifs

À la fin de ce TD, l'étudiant doit pouvoir :

- expliquer la fonction d'un middleware ;
- créer un middleware personnalisé ;
- modifier une requête avant le contrôleur ;
- modifier une réponse après le contrôleur ;
- utiliser un middleware paramétré ;
- comprendre l'ordre d'exécution du pipeline HTTP ;
- enregistrer un alias de middleware.

2.1. 1. Rappel conceptuel

Une requête peut suivre :

$$Request \rightarrow Middleware_A \rightarrow Middleware_B \rightarrow Controller$$

puis :

$$Controller \rightarrow Middleware_B \rightarrow Middleware_A \rightarrow Response$$

Le middleware peut donc intervenir avant et après le contrôleur.

2.2. 2. Créer un middleware de mesure

Exécuter :

```
php artisan make:middleware LogRequestDuration
```

Modifier la classe :

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Symfony\Component\HttpFoundation\Response;

class LogRequestDuration
{
    public function handle(
        Request $request,
        Closure $next
    ): Response {

        $start = microtime(true);

        $response = $next($request);

        $duration =
            microtime(true) - $start;

        Log::info(
```

```
            'HTTP request duration',
            [
                'method' =>
                    $request->method(),

                'path' =>
                    $request->path(),

                'duration_ms' =>
                    round(
                        $duration * 1000,
                        2
                    ),
            ]
        );

        return $response;
    }
}
```

2.3. 3. Utiliser le middleware

Appliquer le middleware sur quelques routes.
Exemple :

```
Route::middleware(
    LogRequestDuration::class
)->group(function () {

    Route::get(
        '/products',
        [ProductController::class, 'index']
    );

});
```

Tester puis examiner :

`storage/logs/laravel.log`

2.4. 4. Ajouter un header à la réponse

Modifier le middleware :

```
$response->headers->set(
    'X-Response-Time',
    round($duration * 1000, 2) . 'ms'
);
```

Observer les headers HTTP dans les outils de développement du navigateur.

2.5. 5. Middleware paramétré

Créer :

```
php artisan make:middleware EnsureWarehouseAccess
```

Structure :

```php
public function handle(
    Request $request,
    Closure $next,
    string $requiredRole
): Response {

    $user = $request->user();

    abort_if(
        ! $user,
        401
    );

    abort_unless(
        $user->role === $requiredRole,
        403
    );

    return $next($request);
}
```

2.6. 6. Alias

Dans la configuration des middleware de l'application, déclarer un alias logique.
Exemple conceptuel :

```php
$middleware->alias([
    'warehouse.role' =>
        \App\Http\Middleware\
        EnsureWarehouseAccess::class,
]);
```

Puis utiliser :

```php
Route::middleware(
    'warehouse.role:manager'
)->group(function () {

    // routes reservees

});
```

2.7. 7. Observer l'ordre

Créer deux middleware :

A et B

Chacun écrit dans le log :

Avant A, Après A, Avant B, Après B

Déterminer l'ordre réellement obtenu.

Questions d'analyse

Pourquoi l'ordre observé est-il souvent :

$$Avant_A \to Avant_B \to Controller \to Après_B \to Après_A$$

et non :

$$Avant_A \to Avant_B \to Controller \to Après_A \to Après_B$$

?

2.8. 8. Travail autonome

Travail en autonomie

Créer un middleware :

`EnsureMaintenanceWindow`

qui bloque certaines opérations entre deux heures configurables, sauf pour les administrateurs. Le middleware doit :

- retourner un statut approprié ;
- produire un message explicite ;
- laisser passer les administrateurs ;
- être testable.

3. TD 17 – Service Container et Service Providers

Objectifs

L'étudiant doit pouvoir :

- expliquer l'inversion de dépendances ;
- distinguer interface et implémentation ;
- utiliser `bind()` ;
- utiliser `singleton()` ;
- remplacer une implémentation ;
- comprendre le rôle d'un Service Provider ;
- utiliser le conteneur dans les tests.

3.1. 1. Cas d'étude : paiement

Nous souhaitons pouvoir utiliser plusieurs systèmes de paiement.
Créer :

`app/Contracts/PaymentGateway.php`

```php
<?php

namespace App\Contracts;

interface PaymentGateway
{
    public function charge(
        int $amount,
        string $reference
    ): string;
}
```

3.2. 2. Première implémentation

Créer :

`app/Services/FakePaymentGateway.php`

```php
<?php

namespace App\Services;

use App\Contracts\PaymentGateway;

class FakePaymentGateway
    implements PaymentGateway
{
    public function charge(
        int $amount,
        string $reference
    ): string {

        return 'FAKE-' . $reference;
    }
}
```

3.3. 3. Binding

Dans un Service Provider :

```
use App\Contracts\PaymentGateway;
use App\Services\FakePaymentGateway;

public function register(): void
{
    $this->app->bind(
        PaymentGateway::class,
        FakePaymentGateway::class
    );
}
```

3.4. 4. Injection

Dans un service métier :

```
public function __construct(
    private PaymentGateway $gateway
) {
}
```

Puis :

```
$transactionId =
    $this->gateway->charge(
        $amount,
        $order->reference
    );
```

3.5. 5. Seconde implémentation

Créer :

`MobileMoneyGateway`

qui implémente la même interface.
Ne pas modifier :

— le contrôleur ;

— le service métier ;

— l'interface.

Modifier uniquement le binding.

Point de contrôle

L'application doit pouvoir changer de fournisseur sans modifier le code métier appelant l'interface.

3.6. 6. bind ou singleton ?

Tester :

```
$this->app->singleton(
    PaymentGateway::class,
    FakePaymentGateway::class
);
```

Comparer conceptuellement avec `bind()`.

Questions d'analyse

Dans quelles situations une même instance doit-elle être partagée ?
Dans quelles situations une nouvelle instance est-elle préférable ?

3.7. 7. Résolution manuelle

Tester :

```
$gateway = app(
    PaymentGateway::class
);
```

Puis :

```
$gateway2 = resolve(
    PaymentGateway::class
);
```

Comparer avec l'injection par constructeur.

3.8. 8. Remplacement en test

Dans un test :

```
$fake = new FakePaymentGateway();

$this->app->instance(
    PaymentGateway::class,
    $fake
);
```

Questions d'analyse

Pourquoi cela améliore-t-il la testabilité de l'application ?

3.9. 9. Défi

Défi avancé

Créer deux implémentations :

- `EmailNotificationSender` ;
- `SmsNotificationSender`.

Les deux implémentent :

`NotificationSender`

Le service métier ne doit dépendre que de l'interface.

4. TD 18 – Events, Listeners et Observers

Objectifs

L'étudiant doit pouvoir :

- distinguer action principale et effet secondaire ;
- créer un Event ;
- créer plusieurs Listeners ;
- déclencher un événement ;
- observer un modèle ;
- comparer Event et Observer ;
- réduire le couplage entre composants.

4.1. 1. Problème

Après création d'une commande, l'application doit :

1. enregistrer la commande ;
2. envoyer une confirmation ;
3. mettre à jour des statistiques ;
4. générer une facture ;
5. écrire un journal.

Écrire toutes ces opérations dans le contrôleur crée un fort couplage.

4.2. 2. Créer un Event

```
php artisan make:event OrderPlaced
```

Exemple :

```
<?php

namespace App\Events;

use App\Models\Order;

class OrderPlaced
{
    public function __construct(
        public Order $order
    ) {
    }
}
```

4.3. 3. Déclencher

Après la transaction métier :

```
use App\Events\OrderPlaced;

event(
    new OrderPlaced($order)
);
```

4.4. 4. Listener

Créer :

```
php artisan make:listener \
UpdateSalesStatistics \
--event=OrderPlaced
```

Exemple :

```
public function handle(
    OrderPlaced $event
): void {

    logger()->info(
        'Order placed',
        [
            'order_id' =>
                $event->order->id,
        ]
    );
}
```

4.5. 5. Plusieurs listeners

Créer :

- `SendOrderConfirmation` ;
- `GenerateInvoice` ;
- `UpdateSalesStatistics`.

Vérifier les événements/listeners connus par Laravel :

```
php artisan event:list
```

4.6. 6. Observer

Créer :

```
php artisan make:observer \
ProductObserver --model=Product
```

Exemple :

```
public function updated(
    Product $product
): void {

    if (
        $product->wasChanged('stock')
    ) {
        logger()->info(
            'Product stock changed',
            [
                'product_id' =>
                    $product->id,

                'stock' =>
                    $product->stock,
            ]
        );
    }
}
```

Enregistrer l'observer dans un Service Provider :

```
Product::observe(
    ProductObserver::class
);
```

4.7. 7. Comparaison

Event métier	Observer Eloquent
Exprime une intention métier	Réagit au cycle de vie du modèle
Peut regrouper plusieurs modèles	Centré sur un modèle
Déclenché explicitement	Déclenché par événement Eloquent
Très lisible métier	Pratique pour réactions techniques

4.8. 8. Travail autonome

Travail en autonomie

Lorsqu'un stock passe sous un seuil configurable :

1. détecter le changement ;
2. produire un événement approprié ;
3. déclencher un listener ;
4. journaliser l'alerte ;
5. ne pas placer toute la logique dans le contrôleur.

5. TD 19 – Jobs, Queues et traitement asynchrone

Objectifs

L'étudiant doit pouvoir :

- distinguer traitement synchrone et asynchrone ;
- créer un Job ;
- dispatcher un Job ;
- configurer une queue ;
- utiliser un worker ;
- gérer les tentatives et erreurs ;
- comprendre les limites d'un traitement asynchrone.

5.1. 1. Problème

Supposons :

$$Création\ facture = 4s$$

et :

$$Envoi\ email = 2s$$

Si tout est effectué avant la réponse HTTP :

$$Request \rightarrow 6s \rightarrow Response$$

L'utilisateur attend inutilement.

5.2. 2. Créer un Job

```
php artisan make:job GenerateInvoice
```

Exemple :

```php
<?php

namespace App\Jobs;

use App\Models\Order;
use Illuminate\Contracts\Queue\ShouldQueue;

class GenerateInvoice
    implements ShouldQueue
{
    public int $tries = 3;

    public function __construct(
        public Order $order
    ) {
    }

    public function handle(): void
    {
        logger()->info(
            'Generating invoice',
```

```
            [
                'order_id' =>
                    $this->order->id,
            ]
        );

        sleep(3);

        // generation de facture
    }
}
```

5.3. 3. Dispatcher

```
GenerateInvoice::dispatch(
    $order
);
```

5.4. 4. Driver database

Dans `.env` :

```
QUEUE_CONNECTION=database
```

Si les tables nécessaires n'existent pas encore, les créer selon la configuration de la version Laravel installée, puis exécuter les migrations.

5.5. 5. Worker

Lancer :

```
php artisan queue:work
```

Tester ensuite une commande.
Observer :

- la réponse HTTP ;
- le terminal du worker ;
- les logs.

5.6. 6. Provoquer un échec

Dans le job :

```
throw new \RuntimeException(
    'Erreur volontaire'
);
```

Lancer :

```
php artisan queue:work --tries=3
```

Observer les tentatives.

5.7. 7. Méthode failed()

Ajouter :

```
public function failed(
    \Throwable $exception
): void {

    logger()->error(
        'Invoice job failed',
        [
            'order_id' =>
                $this->order->id,

            'message' =>
                $exception->getMessage(),
        ]
    );
}
```

5.8. 8. Analyse

Questions d'analyse

1. La commande doit-elle échouer si l'email échoue ?
2. Une facture peut-elle être générée deux fois ?
3. Que se passe-t-il si le worker est arrêté ?
4. Quelles données doivent être transmises au Job ?
5. Pourquoi ne faut-il pas considérer une queue comme une garantie automatique d'exécution unique ?

5.9. 9. Travail autonome

Travail en autonomie

Créer :

— `SendOrderConfirmation` ;

— `GenerateInvoice` ;

— `UpdateDailyStatistics`.

Les trois traitements doivent être exécutés hors de la requête HTTP principale.

6. TD 20 – Cache et optimisation des performances

Objectifs

L'étudiant doit pouvoir :

- mesurer avant d'optimiser ;
- mettre en cache un calcul coûteux ;
- invalider un cache ;
- identifier N+1 ;
- utiliser eager loading ;
- limiter les colonnes chargées ;
- raisonner sur les index SQL.

6.1. 1. Dashboard

Créer un tableau de bord affichant :

- nombre de produits ;
- valeur totale du stock ;
- nombre de commandes ;
- chiffre d'affaires ;
- cinq produits les plus vendus.

6.2. 2. Première version

Exemple :

```
$totalProducts =
    Product::count();

$stockValue =
    Product::selectRaw(
        'SUM(price * stock) as total'
    )->value('total');

$orderCount =
    Order::count();

$revenue =
    Order::sum('total');
```

Mesurer le temps de réponse.

6.3. 3. Cache

Utiliser :

```
use Illuminate\Support\Facades\Cache;

$stats = Cache::remember(
    'dashboard.stats',
    now()->addMinutes(10),
    function () {

        return [
            'products' =>
```

```
                Product::count(),

            'stock_value' =>
                Product::selectRaw(
                    'SUM(price * stock) as total'
                )->value('total'),

            'orders' =>
                Order::count(),

            'revenue' =>
                Order::sum('total'),
        ];
    }
);
```

Comparer les temps.

6.4. 4. Invalidation

Après modification importante du stock :

```
Cache::forget(
    'dashboard.stats'
);
```

Questions d'analyse

Pourquoi un cache rapide mais contenant une information obsolète peut-il être plus dangereux qu'une page légèrement lente ?

6.5. 5. N+1

Comparer :

```
$orders = Order::all();
```

avec :

```
$orders = Order::with([
    'user',
    'items.product',
])->get();
```

6.6. 6. Limiter les colonnes

Tester :

```
Product::query()
    ->select([
        'id',
        'name',
        'price',
        'stock',
    ])
    ->get();
```

6.7. 7. Index SQL

Considérer une requête fréquente :

```
SELECT *
FROM products
WHERE sku = ?
```

Puis :

```
SELECT *
FROM orders
WHERE user_id = ?
ORDER BY created_at DESC
```

Réfléchir aux colonnes qui peuvent bénéficier d'un index.

6.8. 8. Travail autonome

Travail en autonomie

Optimiser une page volontairement lente.
Le rapport doit présenter :

1. temps initial ;
2. nombre de requêtes observées ;
3. problèmes identifiés ;
4. corrections ;
5. temps final ;
6. limites de l'optimisation.

7. TD 21 – Sécurité applicative Laravel

Objectifs

L'étudiant doit pouvoir :

- identifier plusieurs classes de vulnérabilités ;
- distinguer validation et autorisation ;
- éviter le mass assignment non maîtrisé ;
- limiter XSS et CSRF ;
- sécuriser un upload ;
- contrôler les paramètres de tri ;
- utiliser le rate limiting ;
- utiliser des URLs signées.

7.1. 1. Audit initial

Examiner :

```
public function store(Request $request)
{
    $product = Product::create(
        $request->all()
    );

    return $product;
}
```

Identifier les risques.

7.2. 2. Validation

Utiliser :

```
$data = $request->validate([
    'name' =>
        ['required', 'string', 'max:150'],

    'price' =>
        ['required', 'numeric', 'min:0'],

    'stock' =>
        ['required', 'integer', 'min:0'],
]);
```

Puis :

```
Product::create($data);
```

7.3. 3. Mass assignment

Imaginer que la table contient :

`is_approved`

mais que ce champ ne doit jamais être fourni par un utilisateur ordinaire.
Tester conceptuellement :

```
is_approved=1
```

dans une requête modifiée manuellement.

7.4. 4. XSS

Comparer :

```
{{ $product->name }}
```

avec une sortie HTML non échappée.

Attention

Une sortie non échappée ne doit être utilisée que lorsque l'application maîtrise réellement le contenu HTML affiché.

7.5. 5. CSRF

Vérifier la présence de :

```
@csrf
```

dans les formulaires d'écriture.
Répondre :

Questions d'analyse

Pourquoi la connaissance de l'URL ne suffit-elle pas à protéger une opération POST ?

7.6. 6. Tri dynamique dangereux

Éviter :

```
Product::orderBy(
    $request->query('sort')
)->get();
```

Utiliser une liste blanche :

```
$allowed = [
    'name',
    'price',
    'stock',
];

$sort =
    $request->query(
        'sort',
        'name'
    );

if (! in_array(
    $sort,
    $allowed,
    true
)) {
    $sort = 'name';
}

$products =
```

```
    Product::orderBy($sort)->get();
```

7.7. 7. Upload sécurisé

Validation :

```
$request->validate([
    'document' => [
        'required',
        'file',
        'mimes:pdf',
        'max:2048',
    ],
]);
```

Stockage :

```
$path =
    $request
        ->file('document')
        ->store('documents');
```

7.8. 8. Rate limiting

Protéger une route sensible par limitation de fréquence.
Exemple pédagogique :

```
Route::middleware(
    'throttle:10,1'
)->post(
    '/orders',
    [OrderController::class, 'store']
);
```

7.9. 9. URL signée

Créer une URL temporaire :

```
use Illuminate\Support\Facades\URL;

$url =
    URL::temporarySignedRoute(
        'orders.download',
        now()->addMinutes(15),
        [
            'order' => $order->id,
        ]
    );
```

Protéger la route par :

```
->middleware('signed')
```

7.10. 10. Travail autonome

Travail en autonomie

Auditer le projet complet selon :

- entrées utilisateur ;
- autorisations ;
- CSRF ;
- XSS ;
- uploads ;
- secrets ;
- données modifiables ;
- endpoints sensibles ;
- erreurs trop détaillées ;
- exposition des fichiers.

Produire un tableau : problème, risque, correction.

8. TD 22 – API authentifiée par token

Objectifs

L'étudiant doit pouvoir :

- protéger une API ;
- créer un token personnel ;
- transmettre un Bearer token ;
- tester une capacité du token ;
- distinguer 401 et 403 ;
- révoquer un token ;
- appliquer un rate limiting.

8.1. 1. Préparation

Si l'API Laravel et l'authentification par token ne sont pas encore installées, préparer le projet selon la version Laravel utilisée.

Le projet doit disposer d'un modèle `User` capable de créer des tokens.

8.2. 2. Création d'un token

Exemple :

```
$token =
    $user->createToken(
        'mobile-app',
        [
            'products:read',
            'orders:create',
        ]
    )->plainTextToken;
```

Afficher le token une seule fois.

8.3. 3. Appel HTTP

Exemple :

```
curl \
-H "Authorization: Bearer TOKEN_ICI" \
-H "Accept: application/json" \
http://localhost:8000/api/products
```

8.4. 4. Route protégée

Exemple :

```
Route::middleware(
    'auth:sanctum'
)->get(
    '/products',
    [ProductController::class, 'index']
);
```

8.5. 5. Vérifier une capacité

Dans une action :

```
abort_unless(
    $request->user()
        ->tokenCan('orders:create'),
    403
);
```

8.6. 6. Révoquer

Révoquer le token courant :

```
$request->user()
    ->currentAccessToken()
    ->delete();
```

8.7. 7. Scénarios

Tester :

1. aucun token ;
2. token invalide ;
3. token valide ;
4. token sans permission ;
5. token révoqué.

Associer les statuts HTTP.

8.8. 8. Défi

Défi avancé

Créer deux profils :

— application mobile client ;
— application d'administration.

Le token client peut :

— lire les produits ;
— créer une commande.

Le token administration peut également modifier les produits.

9. TD 23 – Concurrence et cohérence du stock

Objectifs

L'étudiant doit pouvoir :

- définir une race condition ;
- expliquer pourquoi une transaction ne résout pas automatiquement tous les conflits ;
- utiliser `lockForUpdate()` ;
- préserver la cohérence du stock ;
- raisonner sur deux requêtes concurrentes.

9.1. 1. Situation

Stock :

$$1$$

Deux utilisateurs lisent simultanément :

$$stock = 1$$

Puis chacun commande une unité.
Résultat incorrect possible :

$$2 \textit{ commandes}$$

pour :

$$1 \textit{ seule unité}$$

9.2. 2. Code naïf

```php
$product =
    Product::findOrFail($id);

if ($product->stock < 1) {
    throw new \Exception(
        'Stock insuffisant'
    );
}

$product->stock--;

$product->save();
```

Questions d'analyse

Pourquoi ce code peut-il être correct avec un seul utilisateur mais incorrect sous concurrence ?

9.3. 3. Transaction avec verrou

```php
DB::transaction(
    function () use ($id) {

        $product =
            Product::query()
                ->lockForUpdate()
                ->findOrFail($id);

        if ($product->stock < 1) {
            throw new \RuntimeException(
                'Stock insuffisant'
            );
        }

        $product->decrement(
            'stock',
            1
        );
    }
);
```

9.4. 4. Analyse temporelle

Compléter :

Temps	Client A	Client B
T1	Lecture/verrou	
T2	Vérification	Tentative lecture
T3	Décrément	Attend
T4	Commit	Reprend
T5		Vérification stock

9.5. 5. Quantité supérieure à 1

Adapter pour :

```php
int $quantity
```

La condition doit être :

```php
if (
    $product->stock < $quantity
) {
    // echec
}
```

9.6. 6. Travail autonome

Travail en autonomie

Simuler deux commandes concurrentes.
Objectifs :

- reproduire le problème ;
- proposer une solution ;
- vérifier qu'aucun stock ne devient négatif ;
- documenter la chronologie.

10. TD 24 – Idempotence et robustesse d'une API

Objectifs

L'étudiant doit pouvoir :

- expliquer l'idempotence ;
- distinguer duplication et concurrence ;
- utiliser une clé d'idempotence ;
- empêcher la création répétée ;
- comprendre les problèmes de retry réseau.

10.1. 1. Situation

Un client envoie :

`POST /api/orders`

La commande est créée, mais la connexion réseau se coupe avant réception de la réponse.
Le client réessaie.
Sans mécanisme spécifique :

$$Order_1 + Order_2$$

peuvent être créées.

10.2. 2. Header

Le client envoie :

```
Idempotency-Key: 8f14e45fceea...
```

La même intention doit réutiliser la même clé.

10.3. 3. Table

Créer une table pédagogique :

```php
Schema::create(
    'idempotency_keys',
    function (Blueprint $table) {

        $table->id();

        $table->foreignId('user_id')
              ->constrained();

        $table->string('key');

        $table->foreignId('order_id')
              ->nullable()
              ->constrained();

        $table->timestamps();

        $table->unique([
            'user_id',
            'key',
```

```
        ]);
    }
);
```

10.4. 4. Algorithme

Pseudo-code :

```
// 1. lire Idempotency-Key

// 2. verifier si la cle existe
// pour cet utilisateur

// 3. si oui :
// retourner la commande precedente

// 4. sinon :
// creer la commande dans une transaction

// 5. enregistrer la cle

// 6. retourner la commande
```

10.5. 5. Questions difficiles

Questions d'analyse

1. Que se passe-t-il si deux requêtes identiques arrivent exactement ensemble ?
2. Pourquoi faut-il une contrainte UNIQUE ?
3. Combien de temps conserver les clés ?
4. Une même clé peut-elle être réutilisée avec un body différent ?
5. Que doit retourner l'API lors d'un retry ?

10.6. 6. Défi

Défi avancé

Empêcher qu'une même clé soit utilisée avec deux payloads différents.
Une possibilité consiste à enregistrer également un hash de la requête.

11. TD 25 – Tests avancés, Fakes et Mocks

Objectifs

L'étudiant doit pouvoir :

- isoler un composant ;
- utiliser Event Fake ;
- utiliser Queue Fake ;
- utiliser Notification Fake ;
- remplacer une dépendance ;
- tester un rollback ;
- tester des effets secondaires.

11.1. 1. Queue Fake

```php
use Illuminate\Support\Facades\Queue;

Queue::fake();

$order = Order::factory()->create();

GenerateInvoice::dispatch(
    $order
);

Queue::assertPushed(
    GenerateInvoice::class
);
```

11.2. 2. Vérifier le contenu

```php
Queue::assertPushed(
    GenerateInvoice::class,
    function ($job) use ($order) {

        return
            $job->order->is(
                $order
            );
    }
);
```

11.3. 3. Event Fake

```php
use Illuminate\Support\Facades\Event;

Event::fake();

event(
    new OrderPlaced($order)
);

Event::assertDispatched(
```

```
    OrderPlaced::class
);
```

11.4. 4. Notification Fake

```
use Illuminate\Support\Facades\Notification;

Notification::fake();

// traitement

Notification::assertSentTo(
    $user,
    OrderConfirmation::class
);
```

11.5. 5. Remplacer PaymentGateway

Créer une fausse implémentation :

```
class FailingPaymentGateway
    implements PaymentGateway
{
    public function charge(
        int $amount,
        string $reference
    ): string {

        throw new RuntimeException(
            'Payment failed'
        );
    }
}
```

Dans le test :

```
$this->app->instance(
    PaymentGateway::class,
    new FailingPaymentGateway()
);
```

11.6. 6. Tester rollback

Après échec :

```
$this->assertDatabaseMissing(
    'orders',
    [
        'reference' =>
            'TEST-ORDER',
    ]
);
```

et :

```
$this->assertSame(
    $initialStock,
    $product->fresh()->stock
);
```

11.7. 7. Travail autonome

Travail en autonomie

Écrire une suite de tests vérifiant :

1. événement de commande déclenché ;
2. job facture dispatché ;
3. notification envoyée ;
4. aucun email réel envoyé en test ;
5. rollback si paiement échoue ;
6. stock restauré après échec ;
7. API idempotente ;
8. utilisateur sans permission obtient 403.

12. TD 26 – Architecture applicative et refactoring

Objectifs

L'étudiant doit pouvoir :

- analyser un contrôleur trop chargé ;
- appliquer le principe de responsabilité unique ;
- utiliser DTO et Value Objects simples ;
- créer une Action métier ;
- distinguer organisation et surarchitecture ;
- justifier un refactoring.

12.1. 1. Contrôleur problématique

Supposons un contrôleur de 250 lignes qui réalise :

- validation ;
- calcul ;
- paiement ;
- stock ;
- création de commande ;
- email ;
- génération PDF ;
- journalisation.

Question :

Quelles responsabilités doivent quitter le contrôleur ?

12.2. 2. Architecture cible

Une possibilité :

$$FormRequest \rightarrow Controller \rightarrow PlaceOrderAction$$

puis :

$$PlaceOrderAction \rightarrow PaymentGateway$$

$$PlaceOrderAction \rightarrow Eloquent$$

$$PlaceOrderAction \rightarrow Event$$

12.3. 3. DTO

Créer :

`OrderData`

```php
<?php

namespace App\Data;

final readonly class OrderData
{
    public function __construct(
        public int $userId,
        public array $items
    ) {
    }
}
```

12.4. 4. Value Object Quantity

```php
<?php

namespace App\ValueObjects;

use InvalidArgumentException;

final readonly class Quantity
{
    public function __construct(
        public int $value
    ) {
        if ($value <= 0) {
            throw new InvalidArgumentException(
                'Quantity must be positive'
            );
        }
    }
}
```

12.5. 5. Action

Créer :

app/Actions/PlaceOrderAction.php

```php
final class PlaceOrderAction
{
    public function __construct(
        private PaymentGateway $gateway
    ) {
    }

    public function execute(
        OrderData $data
    ): Order {

        // transaction
        // stock
        // commande
        // paiement
        // event

```

```
        return $order;
    }
}
```

12.6. 6. Contrôleur final

Le contrôleur doit devenir principalement un adaptateur HTTP :

```
public function store(
    StoreOrderRequest $request,
    PlaceOrderAction $action
) {
    $data = new OrderData(
        userId:
            $request->user()->id,

        items:
            $request->validated('items')
    );

    $order =
        $action->execute($data);

    return to_route(
        'orders.show',
        $order
    );
}
```

12.7. 7. Discussion Repository

Questions d'analyse

Eloquent fournit déjà une abstraction importante de l'accès aux données.
Faut-il créer systématiquement un Repository pour chaque modèle ?
Répondre en argumentant, et non par oui/non.

12.8. 8. Travail autonome

Travail en autonomie

Refactoriser une fonctionnalité existante.
Le rapport doit présenter :

- architecture avant ;
- problèmes identifiés ;
- architecture après ;
- bénéfices ;
- coûts ou complexité ajoutée.

13. TD 27 – Commandes Artisan et Scheduler

Objectifs

L'étudiant doit pouvoir :

- créer une commande Artisan ;
- définir arguments et options ;
- produire un retour CLI ;
- planifier une tâche ;
- distinguer commande et Job ;
- automatiser un traitement périodique.

13.1. 1. Créer une commande

```
php artisan make:command CheckLowStock
```

13.2. 2. Signature

Exemple :

```
protected $signature =
    'stock:check
    {--threshold=5 :
    Seuil de stock faible}';
```

13.3. 3. Exécution

```
public function handle(): int
{
    $threshold =
        (int) $this->option(
            'threshold'
        );

    $products =
        Product::where(
            'stock',
            '<=',
            $threshold
        )->get();

    if ($products->isEmpty()) {

        $this->info(
            'Aucun stock faible.'
        );

        return self::SUCCESS;
    }

    foreach ($products as $product) {

        $this->warn(
            $product->name
```

```
            . ' : '
            . $product->stock
        );
    }

    return self::SUCCESS;
}
```

Tester :

```
php artisan stock:check
```

puis :

```
php artisan stock:check --threshold=10
```

13.4. 4. Scheduler

Dans la configuration de planification du projet :

```
use Illuminate\Support\Facades\Schedule;

Schedule::command(
    'stock:check --threshold=5'
)->dailyAt('07:00');
```

13.5. 5. Tester localement

```
php artisan schedule:list
```

Puis éventuellement :

```
php artisan schedule:work
```

13.6. 6. Travail autonome

Travail en autonomie

Créer :

- `orders:expire` ;
- `reports:daily` ;
- `stock:check`.

Déterminer pour chacun :

- fréquence ;
- traitement synchrone ou Job ;
- logs ;
- risque d'exécution simultanée.

14. TD 28 – Déploiement Laravel sous Linux

Objectifs

L'étudiant doit pouvoir :

- distinguer environnement de développement et production ;
- expliquer le rôle de Nginx/Apache et PHP-FPM ;
- configurer l'environnement Laravel ;
- protéger les fichiers non publics ;
- gérer les permissions ;
- préparer les caches ;
- exécuter les migrations ;
- maintenir un worker de queue.

14.1. 1. Architecture

$$Internet \rightarrow Nginx \rightarrow PHP-FPM \rightarrow Laravel \rightarrow Database$$

Le serveur Web doit exposer :

`public/`

et non la racine complète du projet.

14.2. 2. Dépendances production

Dans le projet :

```
composer install \
--no-dev \
--optimize-autoloader
```

14.3. 3. Environnement

Exemple :

```
APP_ENV=production
APP_DEBUG=false
```

Configurer également :

- base de données ;
- cache ;
- queue ;
- mail ;
- URL de l'application.

14.4. 4. Clé d'application

Si nécessaire :

```
php artisan key:generate
```

Ne jamais régénérer arbitrairement la clé d'une application déjà en production.

14.5. 5. Permissions

Les répertoires devant être inscriptibles par le processus PHP incluent notamment :

`storage/`

et :

`bootstrap/cache/`

14.6. 6. Caches

En production :

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Pour nettoyer :

```
php artisan optimize:clear
```

14.7. 7. Migration

```
php artisan migrate --force
```

Attention

Une migration de production doit être préparée et sauvegardée.
`migrate:fresh` ne doit pas être utilisé sur une base de production à conserver.

14.8. 8. Configuration Nginx simplifiée

Exemple pédagogique :

```
server {
    listen 80;

    server_name example.test;

    root /var/www/app/public;

    index index.php;

    location / {
        try_files
            $uri
            $uri/
            /index.php?$query_string;
    }

    location ~ \.php$ {
        include fastcgi_params;

        fastcgi_param
            SCRIPT_FILENAME
            $document_root$fastcgi_script_name;

        fastcgi_pass
```

```
            unix:/run/php/php-fpm.sock;
    }
}
```

Le chemin exact de PHP-FPM dépend de la distribution.

14.9. 9. Worker

Un worker doit fonctionner même après fermeture du terminal.
En production, il doit être supervisé par un mécanisme système.
Commande de base :

```
php artisan queue:work \
--sleep=3 \
--tries=3
```

14.10. 10. Logs

En cas d'erreur 500, examiner notamment :

— logs Laravel ;
— logs Nginx/Apache ;
— logs PHP-FPM ;
— permissions ;
— variables `.env`.

14.11. 11. Travail autonome

Travail en autonomie

Préparer une procédure de déploiement reproductible contenant :

1. récupération du code ;
2. dépendances ;
3. configuration ;
4. permissions ;
5. migration ;
6. caches ;
7. démarrage worker ;
8. test HTTP ;
9. stratégie de rollback.

15. TD 29 – Broadcasting et temps réel

Objectifs

Ce TD est optionnel.
L'étudiant doit pouvoir :

- expliquer le besoin temps réel ;
- distinguer Event interne et Broadcast Event ;
- créer un canal ;
- diffuser une modification ;
- raisonner sur les autorisations d'un canal privé.

15.1. 1. Scénario

Deux navigateurs affichent la même commande.
Son statut change :

$$pending \rightarrow paid \rightarrow preparing \rightarrow shipped$$

Le deuxième navigateur doit recevoir le changement sans rechargement manuel.

15.2. 2. Event broadcast

Exemple conceptuel :

```
use Illuminate\Contracts\Broadcasting\
ShouldBroadcast;

class OrderStatusChanged
    implements ShouldBroadcast
{
    public function __construct(
        public Order $order
    ) {
    }

    public function broadcastOn(): array
    {
        return [
            new PrivateChannel(
                'orders.'
                . $this->order->id
            ),
        ];
    }
}
```

15.3. 3. Autorisation du canal

Le client ne doit pouvoir écouter que les commandes qu'il est autorisé à consulter.
Pseudo-code :

```
Broadcast::channel(
    'orders.{order}',
    function (
        User $user,
```

```
        Order $order
    ) {
        return
            $user->id ===
                $order->user_id
            ||
            $user->is_admin;
    }
);
```

15.4. 4. Analyse

Questions d'analyse

Quelle différence entre :

- polling ;
- Server-Sent Events ;
- WebSocket ;
- broadcasting Laravel ?

Dans quels cas le temps réel est-il réellement nécessaire ?

16. TD 30 – Incident de production simulé

Objectifs

Ce TD final évalue la capacité à diagnostiquer une application plutôt qu'à appliquer une recette. L'étudiant doit pouvoir :

- reproduire un incident ;
- collecter des indices ;
- formuler des hypothèses ;
- identifier la cause ;
- corriger ;
- écrire un test de non-régression ;
- rédiger un rapport technique.

16.1. 1. Situation

L'application de production présente simultanément les symptômes suivants :

1. le dashboard prend parfois 8 secondes ;
2. certains stocks deviennent négatifs ;
3. une commande peut être créée deux fois ;
4. des factures ne sont parfois pas générées ;
5. un utilisateur normal peut atteindre une URL d'administration ;
6. une API retourne parfois 500 ;
7. certains utilisateurs voient des données de commande obsolètes.

16.2. 2. Méthode de diagnostic

Pour chaque problème :

Symptôme	Hypothèse	Preuve recherchée	Correction

16.3. 3. Dashboard lent

Rechercher notamment :

- N+1 ;
- absence d'index ;
- requêtes inutiles ;
- agrégations coûteuses ;
- cache absent ou mal invalidé.

16.4. 4. Stock négatif

Rechercher :

— absence de transaction ;
— race condition ;
— absence de verrou ;
— validation uniquement côté navigateur.

16.5. 5. Commande doublée

Rechercher :

— double clic ;
— retry client ;
— absence de clé idempotente ;
— absence de contrainte unique.

16.6. 6. Facture absente

Rechercher :

— worker arrêté ;
— Job en échec ;
— limite de tentatives ;
— failed jobs ;
— exception non journalisée.

16.7. 7. Accès administration

Rechercher :

— protection seulement dans Blade ;
— middleware manquant ;
— Policy incorrecte ;
— route non protégée.

16.8. 8. API 500

Analyser :

— logs ;
— stack trace en développement ;
— données de requête ;
— exceptions métier ;
— dépendances externes ;
— réponse attendue.

16.9. 9. Données obsolètes

Rechercher :

— cache ;
— mauvaise invalidation ;
— Job différé ;
— transaction non terminée ;
— navigateur/client.

16.10. 10. Livrable final

Livrable attendu

Le groupe remet :

1. un rapport d'incident ;
2. les causes identifiées ;
3. les corrections effectuées ;
4. les tests ajoutés ;
5. les risques restant ouverts ;
6. une courte présentation technique.

17. Projet de synthèse avancé

Le projet doit intégrer une architecture réaliste.
Fonctionnalités minimales :

— authentification Web ;
— API sécurisée par token ;
— produits et catégories ;
— commandes ;
— contrôle concurrent du stock ;
— transaction ;
— idempotence de création de commande ;
— événement métier ;
— Job en queue ;
— cache ;
— tests automatisés ;
— commande Artisan ;
— tâche planifiée ;
— procédure de déploiement.

18. Architecture cible

Une requête Web ou API peut être représentée par :

$$\textit{Client}$$

$$\downarrow$$

$$\textit{Middleware}$$

$$\downarrow$$

$$\textit{Controller}$$

$$\downarrow$$

$$\textit{FormRequest} + \textit{Policy}$$

$$\downarrow$$

$$\textit{Action/Service}$$

$$\downarrow$$

$$\textit{Domain} + \textit{Eloquent}$$

$$\downarrow$$

$$Database$$

et éventuellement :

$$Event \rightarrow Listener \rightarrow Job \rightarrow Queue$$

Le résultat retourne ensuite :

$$Blade$$

ou :

$$JSON$$

19. Proposition de barème

Critère	Points
Architecture et séparation des responsabilités	3
Sécurité et autorisations	2
Concurrence et cohérence des données	2
Events, Jobs et Queues	2
Cache et performances	2
API sécurisée	2
Tests avancés	3
Automatisation et exploitation	2
Qualité des justifications techniques	2
Total	**20**

20. Questions possibles de soutenance

L'étudiant doit pouvoir répondre sans simplement montrer le code.

1. Pourquoi avez-vous utilisé un middleware plutôt qu'un contrôleur ?
2. Pourquoi cette dépendance est-elle injectée par interface ?
3. Quel est le rôle du Service Container ?
4. Quelle différence entre Event et Job ?
5. Pourquoi utiliser une Queue ?
6. Que se passe-t-il si le Job échoue ?
7. Pourquoi le cache doit-il être invalidé ?
8. Qu'est-ce qu'une race condition ?
9. Pourquoi une transaction seule ne suffit-elle pas toujours ?
10. Quel problème résout `lockForUpdate()` ?
11. Qu'est-ce que l'idempotence ?
12. Pourquoi une API peut-elle recevoir deux fois la même requête ?
13. Quelle différence entre 401 et 403 ?
14. Pourquoi une Policy reste nécessaire même si le bouton est caché dans Blade ?
15. Pourquoi utiliser un Fake dans un test ?

16. Qu'est-ce qu'un test de non-régression ?
17. Pourquoi le dossier `public/` doit-il être la racine Web ?
18. Pourquoi `APP_DEBUG=false` en production ?
19. Pourquoi un worker de queue doit-il être supervisé ?
20. Quelle amélioration apporteriez-vous encore à votre architecture ?

21. Synthèse finale

La progression complète du module peut maintenant être résumée ainsi :

$$PHP \rightarrow Framework \rightarrow MVC$$

$$\downarrow$$

$$Routing \rightarrow Controllers \rightarrow Blade$$

$$\downarrow$$

$$Database \rightarrow Eloquent \rightarrow Relations$$

$$\downarrow$$

$$Validation \rightarrow Authentication \rightarrow Authorization$$

$$\downarrow$$

$$Services \rightarrow Transactions \rightarrow API \rightarrow Tests$$

$$\downarrow$$

$$Middleware \rightarrow Container \rightarrow Events \rightarrow Queues$$

$$\downarrow$$

$$Cache \rightarrow Security \rightarrow Concurrency$$

$$\downarrow$$

$$Idempotence \rightarrow Architecture \rightarrow Deployment$$

Compétence finale attendue

L'étudiant ne doit plus seulement demander :

« Quelle commande Laravel dois-je utiliser ? »

mais plutôt :

« Quel problème dois-je résoudre, quelle responsabilité est concernée, quelles garanties dois-je assurer et comment vérifier que la solution est correcte ? »